Sécuriser l'IoT : Connexion MQTT Sécurisée sur ESP32

Du Cloud HiveMQ
au Microcontrôleur
via SSL/TLS

Objectif Terminal : Sécuriser une connexion MQTT entre un ESP32 et un broker.

Technologie : **MQTT sur port 8883 (TLS)**.

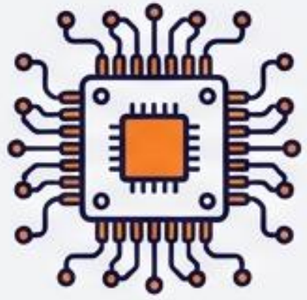
Topologie et Objectifs de l'Atelier

1. Cloud

Création du Cluster HiveMQ

2. Sécurité

Analyse du certificat SSL (Chain of Trust)



3. Hardware

Programmation C++ sur ESP32

- Infrastructure : Déploiement d'un broker Serverless sur HiveMQ Cloud.
- Cryptographie : Intégration du certificat CA Root (Let's Encrypt) pour éviter les attaques MITM.
- Protocole : Utilisation de la librairie WiFiClientSecure.h pour le chiffrement SSL.

Étape 1 : Déploiement du Cluster HiveMQ

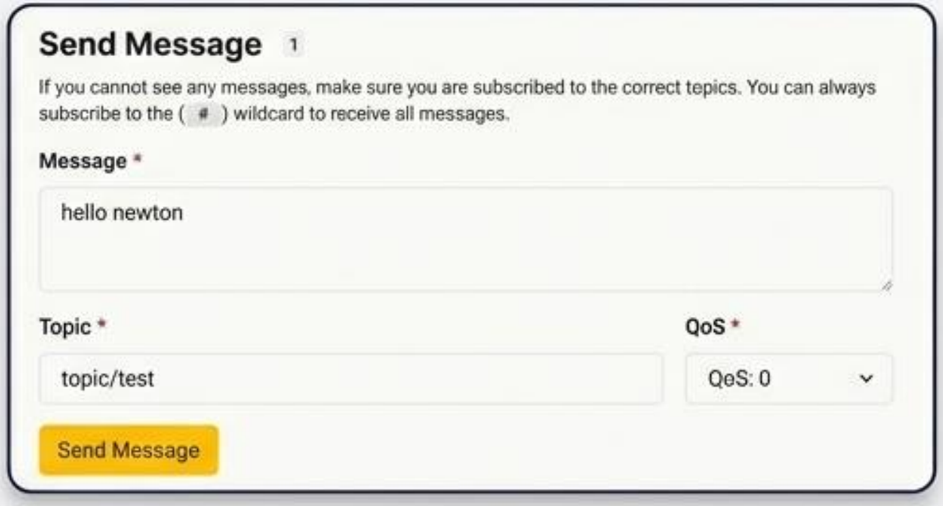
1. Créer un compte sur console.hivemq.cloud
2. Créer un nouveau cluster (Sélectionner AWS ou Azure - sans impact technique).
3. Résultat attendu : Un cluster actif sur le port standard TLS 8883.

Gestion des Accès et Identifiants

Définir un **Username** et un **Password** robustes dans l'onglet **Access Management**.

Validation de l'Infrastructure Cloud

Envoi



Réception

Outil : HiveMQ Web Client
Test : S'abonner au topic `topic/test` et publier un message.
Validation : Si le message apparaît, le broker est opérationnel.

Comprendre la Chaîne de Confiance SSL

Requis par l'ESP32 →

RACINE / ROOT (Depth 2)
ISRG Root X1

INTERMÉDIAIRE (Depth 1)
Let's Encrypt R11

SERVEUR (Depth 0)
*.s1.eu.hivemq.cloud

```
openssl s_client -connect ... -showcerts
```

L'ESP32 ne connaît pas le serveur HiveMQ. Il doit posséder le certificat Racine (Depth 2) pour valider l'authenticité de toute la chaîne.

La Clé de Sécurité : ISRG Root X1

Issuer :

Internet Security Research Group

Rôle :

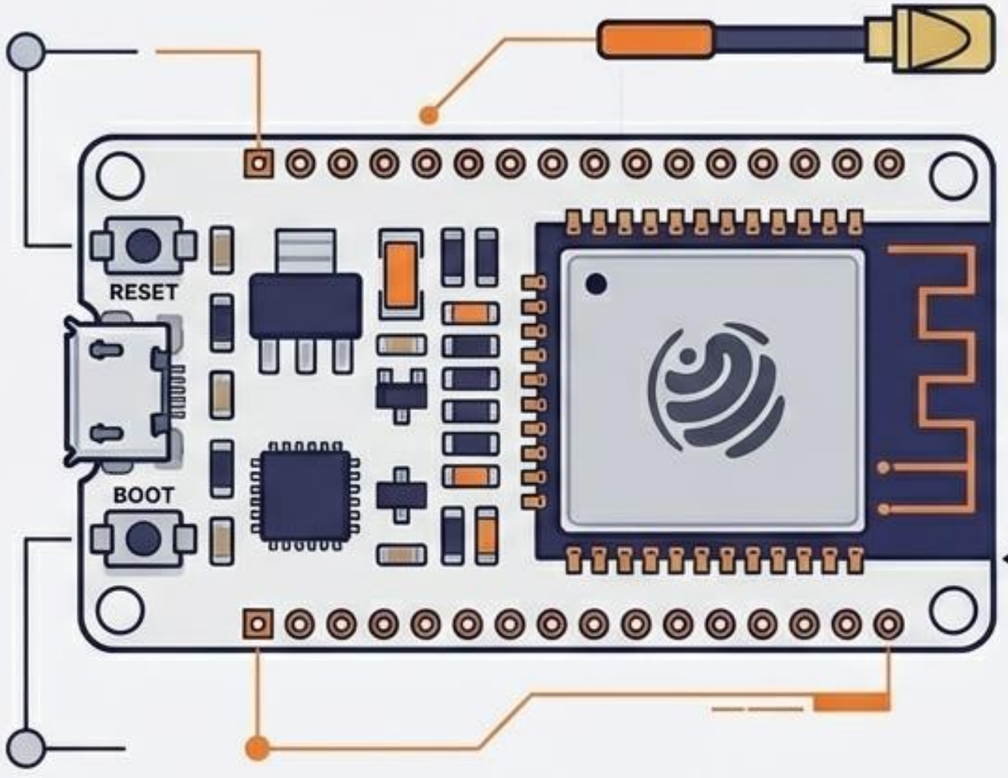
Certificat racine (CA) présent dans la banque de certificats standard (`/etc/ssl/certs/`)

Nécessité :

Le certificat racine n'est pas transmis par le serveur lors du handshake ; il doit être 'hardcodé' ou uploadé sur le microcontrôleur.

NotebookLM

Étape 2 : Préparation de l'ESP32



IDE Arduino
(VS Code / AppImage)

WiFi.h

WiFiClientSecure.h
(SSL/TLS)

PubSubClient.h

Sélectionner la carte « ESP32 Dev Module » dans le Boards Manager.

NotebookLM

Configuration des Variables Globales

```
const char* ssid = "VOTRE_WIFI_SSID";
const char* password = "VOTRE_WIFI_PASSWORD";

// HiveMQ Broker URL
const char* mqtt_server = "xxxxxxx.s1.eu.hivemq.cloud";

// Port Sécurisé (TLS)
const int mqtt_port = 8883;

const char* mqtt_user = "votre_user";
const char* mqtt_pass = "votre_password";
```

Standard pour MQTTs (Secure). Le port 1883 n'est pas utilisé ici.

NotebookLM

Intégration du Certificat CA (C++)

```
// Certificat racine ISRG Root X1 (Let's Encrypt)
const char* ca_cert = \
  "-----BEGIN CERTIFICATE-----\n" \
  "MIIFazCCA1OgAwIBAgIRAIIQz7DSQONZRGPgu2OCiwAwDQYJKoZIhvcNAQELBQAw
  "TzELMAkGA1UEBhMCVVMxKTAnBgNVBAoTIEIudGVybnV0IFN1Y3VyaXR5IFRJc2Vh
  "..."
  "-----END CERTIFICATE-----\n";
```

- On copie le contenu du **ISRG Root X1** dans une variable constante.
- **Pourquoi ?** Cela permet à l'ESP32 de vérifier mathématiquement que le serveur HiveMQ est authentique.
- **Protection** : Bloque les attaques **MITM (Man-In-The-Middle)**.

L'Initialisation Sécurisée (Setup)

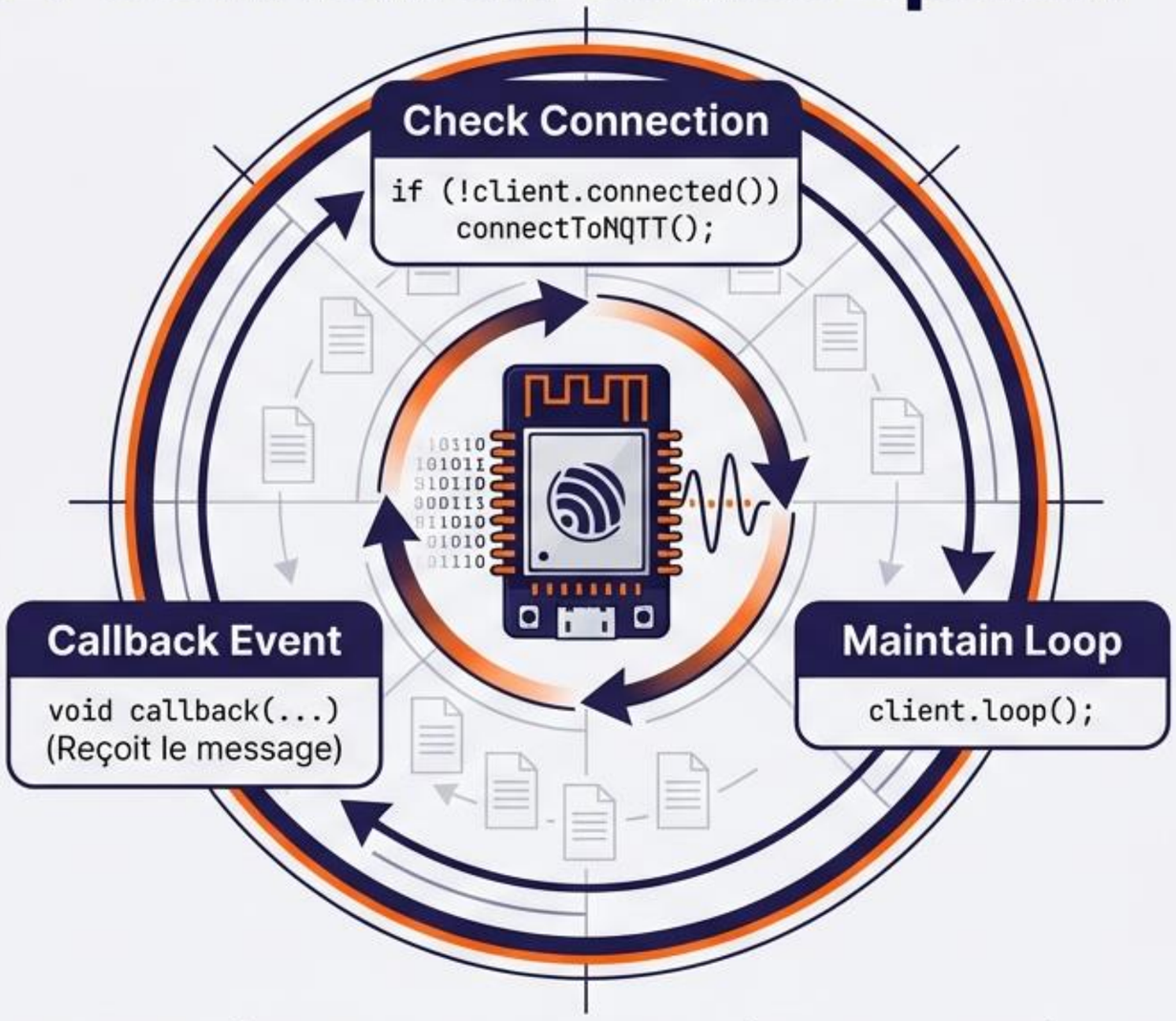
`ca_cert` (Variable) → `wifiClient` (Object) → `PubSubClient` (Object)

```
void setup() {
  Serial.begin(115200);
  connectToWiFi();

  // L'étape CRITIQUE de sécurité
  wifiClient.setCACert(ca_cert);

  client.setServer(mqtt_server, mqtt_port);
  connectToMQTT();
}
```

L'étape **CRITIQUE** : Intègre le certificat CA dans le client WiFi sécurisé, permettant la vérification TLS.

NotebookLM

Cycle de Vie : Connexion et Réception



La fonction **callback** décode le payload caractère par caractère pour l'afficher.

NotebookLM

Test Final : De Mosquitto à l'ESP32

```
> mosquitto_pub -h ...hivemq.cloud -p 8883 -s -t
formation/IOTsecurity -m "Hello IOT security"
```

```
Connexion au WiFi......
Connecté au WiFi
Connecté au broker MQTT
Message reçu sur le sujet : formation/IOTsecurity
Message : Hello IOT security
```

Résumé : Les 3 Piliers du MQTTS

Authentification

Identifiants User/Password gérés par le Broker.

Chiffrement

Transport TLS sur le port 8883 (Tunnel sécurisé).

Vérification

Certificat CA (ISRG Root X1) pour l'identité du serveur.

L'ESP32 est désormais un nœud IoT de confiance.

NotebookLM